

DPA-DP auto-detector evaluation: norm surge and cosine divergence metrics

This experiment tests whether server-side gradient statistics can automatically detect the onset of a simulated disaster during federated training, without any explicit signal from clients.

The setup uses MobileNetV3 trained on the AIDER dataset (five classes: normal, traffic_incident, fire, flooded_areas, collapsed_building) across a 10-client Flower federation with equal-sized participation. Training runs for 30 rounds. For the first 20 rounds, all clients train on normal and traffic_incident images only. From round 21 onward, a disaster is simulated: for half of the 10 clients, a portion of their normal-class images is replaced with an equal-sized mix of fire, flooded_areas, and collapsed_building images, so each affected client's dataset changes while its total size stays constant. The remaining five clients continue training on the unchanged normal and traffic_incident dataset until the end.

Three detector signals are evaluated across this transition - per-client and aggregate gradient norm surge, per-client and aggregate cosine divergence, and cross-client cosine dispersion - under three privacy configurations: no DP-SGD, gradient clipping only, and gradient clipping with gaussian noise. The goal is to characterize which signals are usable as detection cues.

Summary of results

Without differential privacy, all three signal groups show a clear disruption at the disaster round: gradient norm surge and cosine divergence spike sharply for the affected clients, and a secondary, delayed rise appears afterward as unaffected clients continue training against the shifted global model. The same disruption remains visible at a smaller scale once gradient clipping alone is applied.

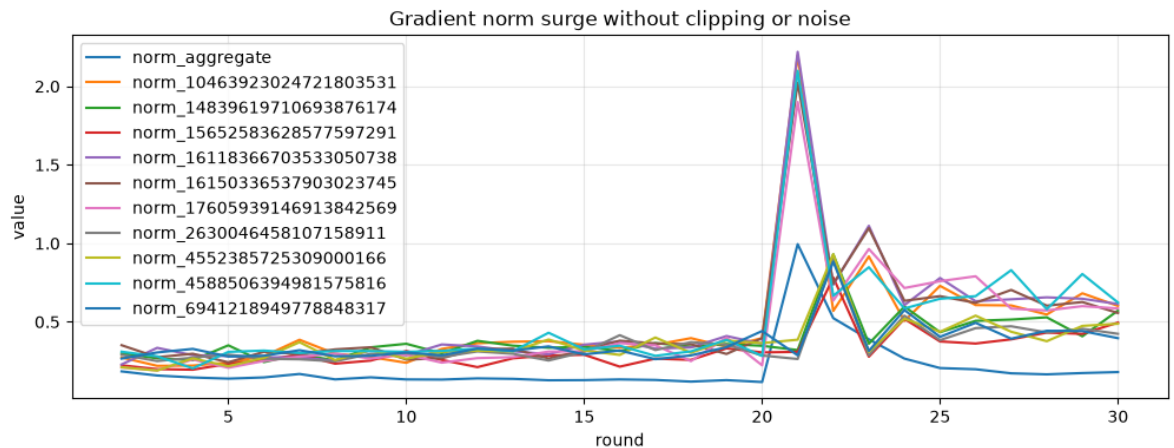
Once gaussian noise is added, gradient norm surge and per-client cosine divergence flatten into the noise floor and the disaster moment is no longer distinguishable. Cross-client cosine dispersion shows an elevated fluctuation around the disaster round, but the elevation is not clearly separated in magnitude from other fluctuations across the rounds.

Detector metrics with DP disabled

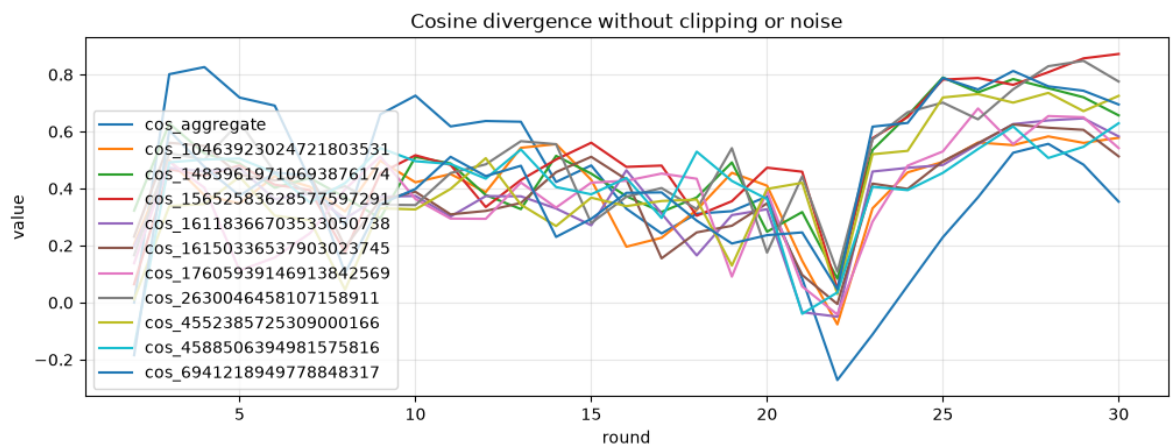
max_grad_norm: inf, noise_multiplier: 0, tune_layers: 1, learning_rate: 0.02

Without differential privacy, all diagrams show a clear spike immediately after the simulated disaster event. The gradient norm and cosine divergence of the aggregated weights show a similar pattern, at a slightly different scale.

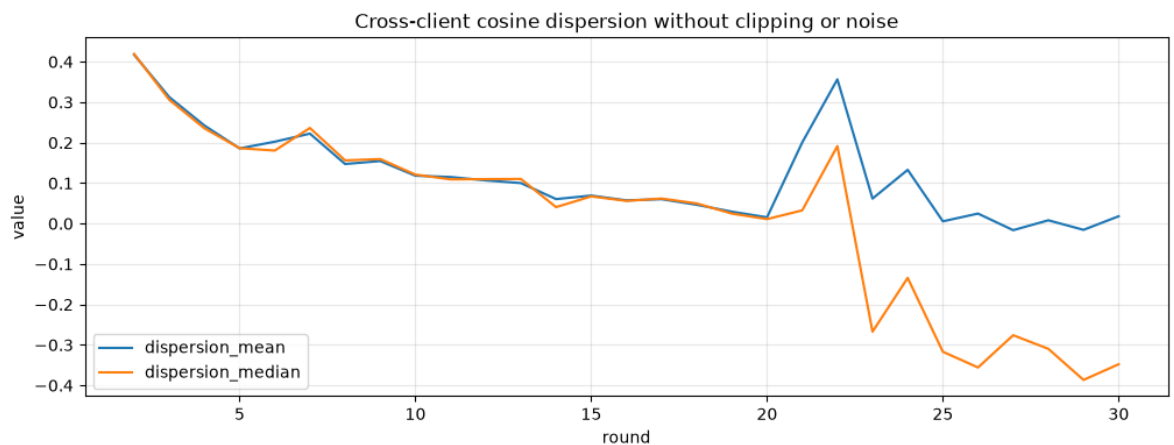
In [2]: `flwr_values("clear_norm", "Gradient norm surge without clipping or noise")`



In [3]: `flwr_values("clear_cos", "Cosine divergence without clipping or noise")`



In [4]: `flwr_values("clear_disp", "Cross-client cosine dispersion without clipping or noise")`

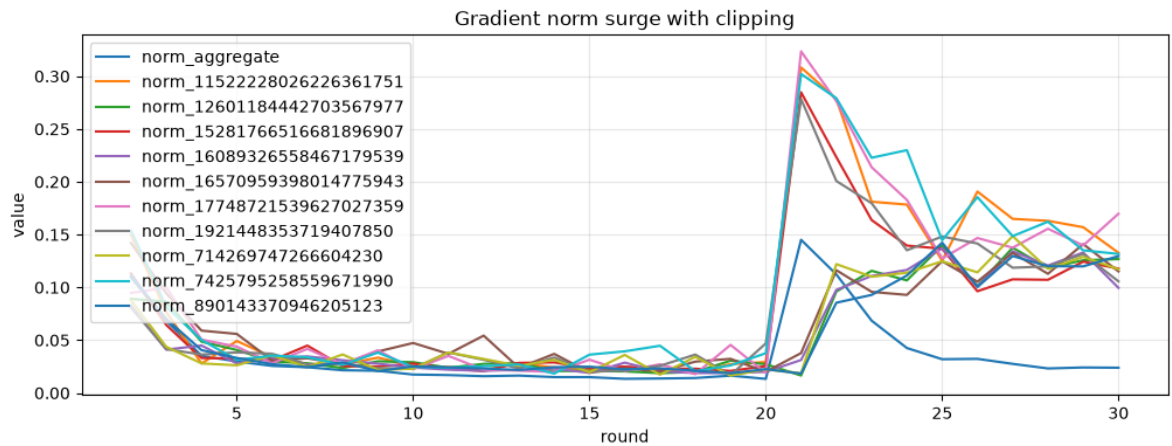


Detector metrics with DP clipping

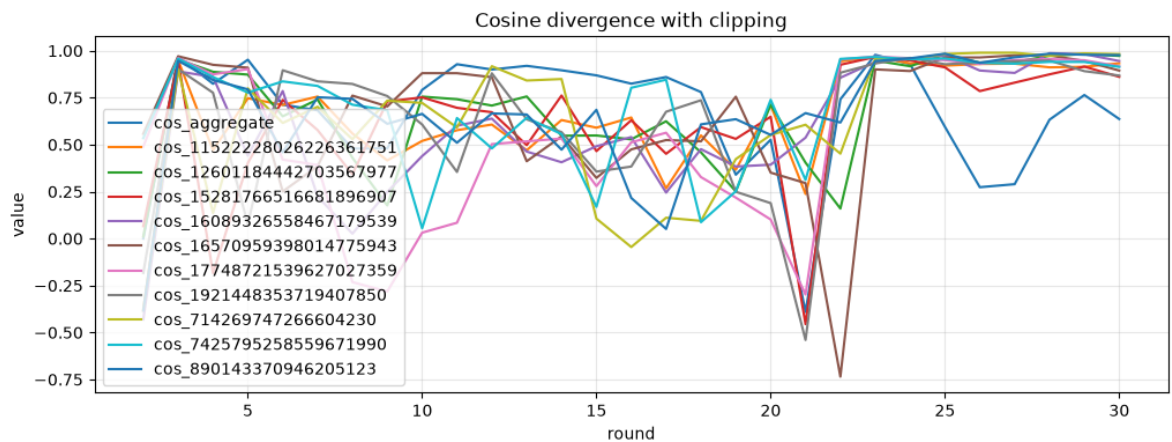
max_grad_norm: 1, noise_multiplier: 0, tune_layers: 1, learning_rate: 0.02

Gradient clipping bounds the very quantity we are trying to observe, so the detected surge is smaller in magnitude than in the previous test. The disaster spike remains clearly visible across all diagrams.

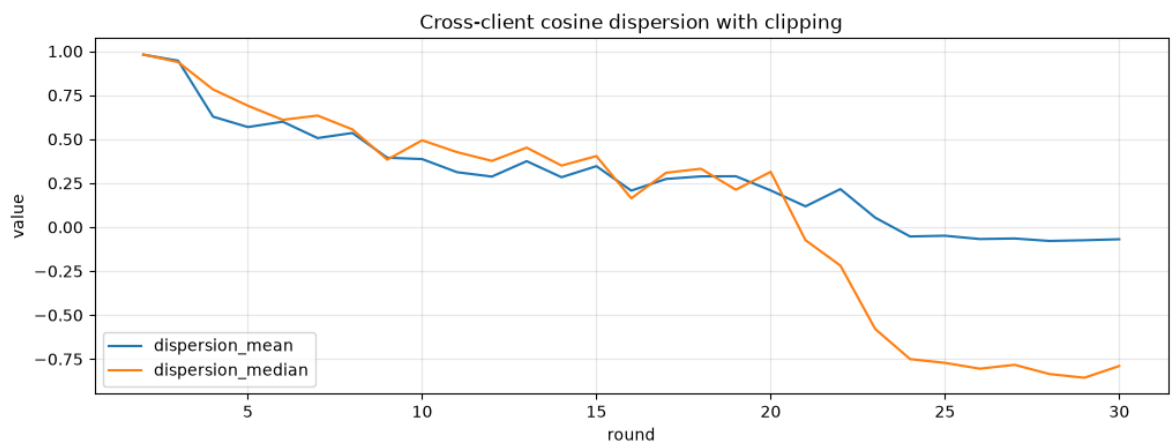
```
In [5]: flwr_values("clip_1_norm", "Gradient norm surge with clipping", loc="uppe
```



```
In [6]: flwr_values("clip_1_cos", "Cosine divergence with clipping")
```



```
In [7]: flwr_values("clip_1_disp", "Cross-client cosine dispersion with clipping")
```

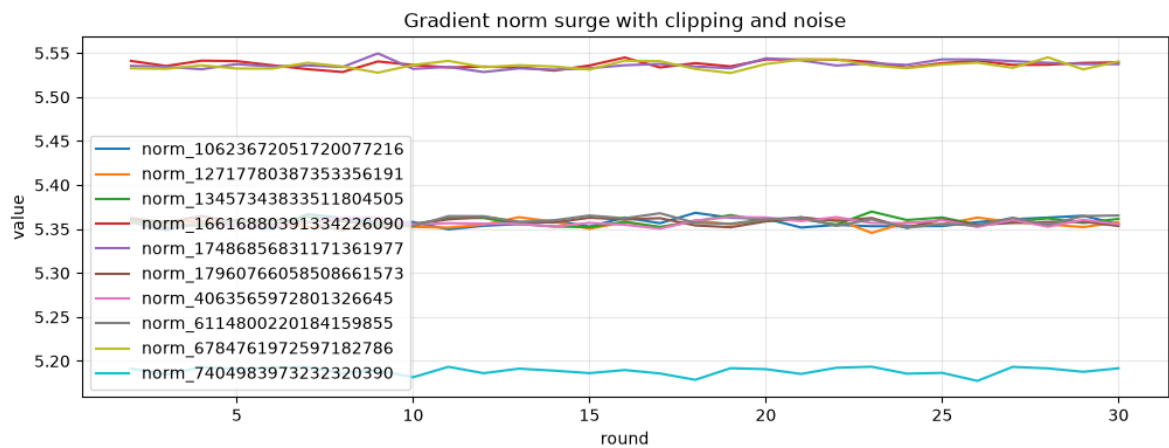


Detector metrics with DP clipping and privacy noise

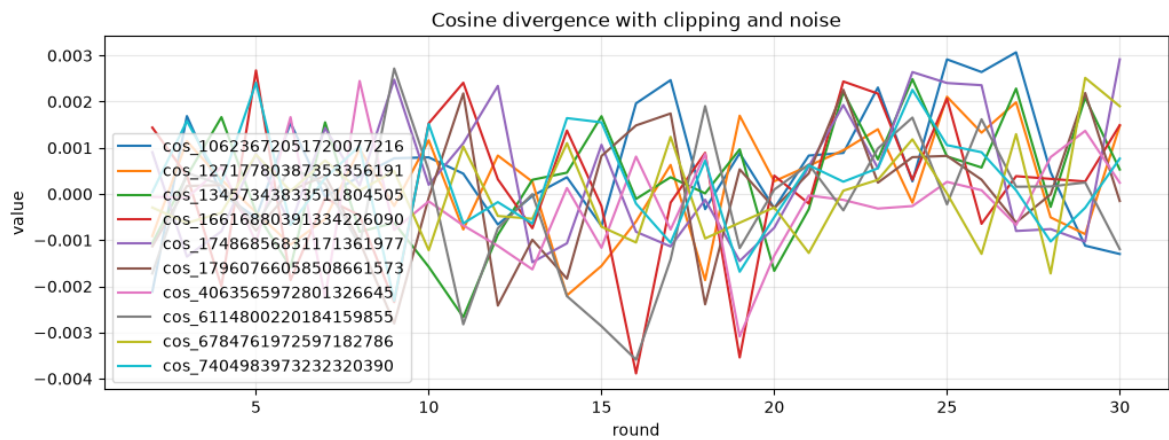
max_grad_norm: 1, noise_multiplier: 0.5, tune_layers: 1, learning_rate: 0.02

Adding Gaussian noise flattens both the gradient norm and cosine divergence diagrams, masking the signal. Cross-client cosine dispersion shows an elevated fluctuation after the disaster moment.

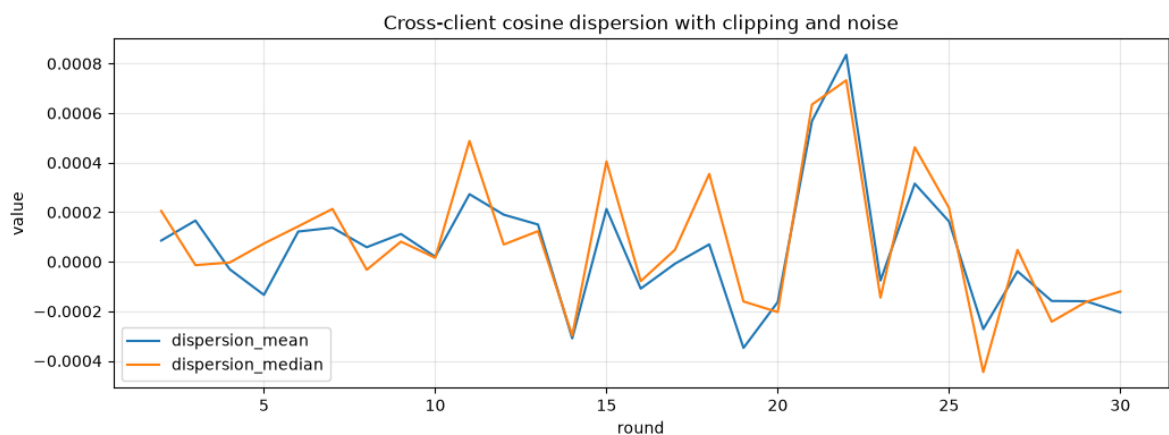
```
In [8]: flwr_values("dp_0.5_norm", "Gradient norm surge with clipping and noise")
```



```
In [9]: flwr_values("dp_0.5_cos", "Cosine divergence with clipping and noise")
```



```
In [10]: flwr_values("dp_0.5_disp", "Cross-client cosine dispersion with clipping")
```



```
In [ ]:
```